

Deploying AI

Optimization and System Design

```
$ echo "Data Sciences Institute"
```

Main Points

Main Points

1. Production AI applications require far more engineering than a working prototype: robust evaluation, guardrails, routing, caching, and feedback loops are all necessary.
2. The AI engineering stack evolves in layers: embedded software → custom API calls → RAG → guardrails → model routing → caching — each layer adds capability and control.
3. Guardrails operate on both inputs (blocking sensitive data or out-of-scope queries) and outputs (catching empty, malformed, toxic, or factually inconsistent responses).
4. Model routers use intent classification to direct queries to the most appropriate model, balancing quality, latency, and cost across a portfolio of models.

Main Points (cont.)

5. Caching — prompt, exact, and semantic — can substantially reduce both latency and cost, especially in applications with long system prompts or repetitive queries.
6. Business metrics must be mapped to AI metrics; systematic experimentation, prompt tuning, and feedback loops are the levers for iterative improvement.
7. In-context learning via RAG reduces model staleness by continuously incorporating new information without requiring retraining.

Agenda

Agenda

- Inference optimization
- AI engineering architecture
- User feedback

Software Products with Embedded AI

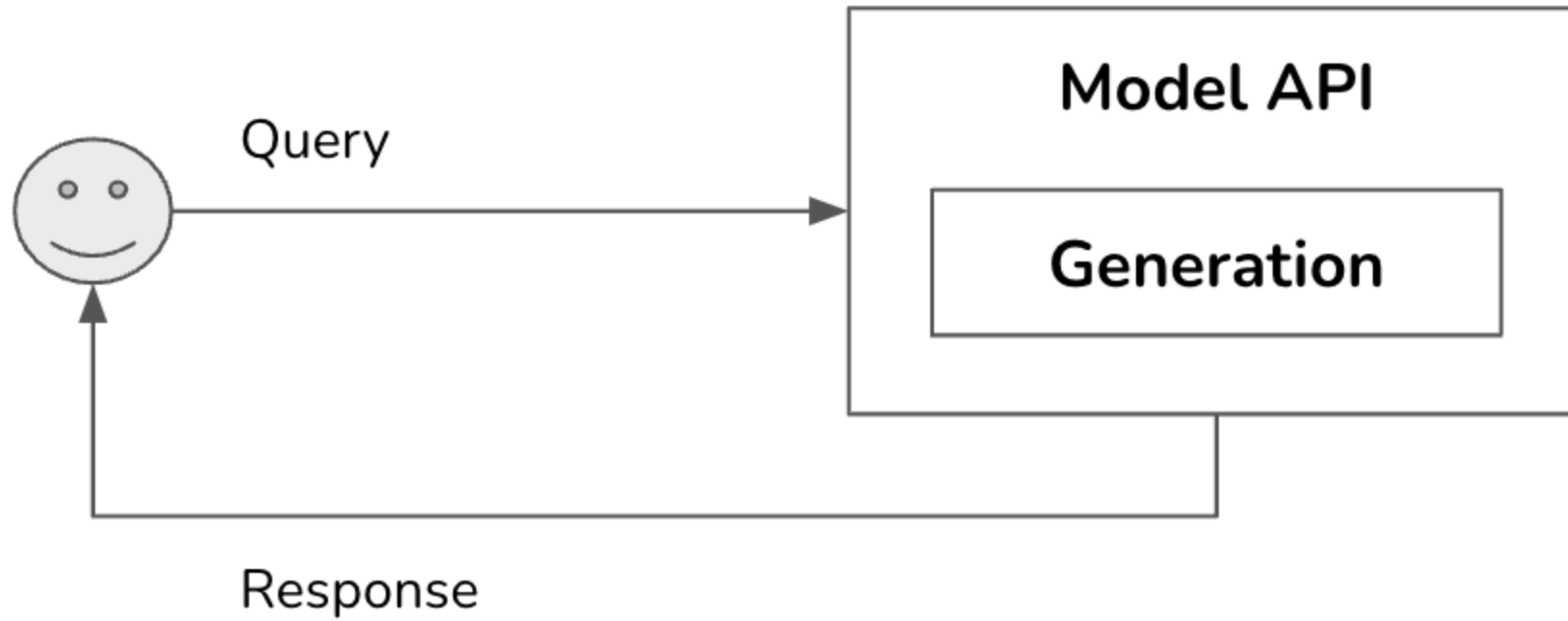
AI models have become generally available through general-purpose software:

- Generate images, illustrations, and video
- Generate and rewrite text
- Summarize documents and calls
- Gather data
- Review and write code

Initial Stage: Interaction through LLMs Embedded in General Purpose Software

- The simplest form of implementation is to acquire software with embedded AI (MS Copilot, ChatGPT, etc).
- Offers a simple mode of operation.
- Low barriers to entry diminish the competitive advantage.
- Guidelines and policies have limited effect over risk exposure.

Initial Stage



Challenges in Building Production-Level Applications with LLM (1/2)

Using LLMs:

- It is easy to create a cool prototype, but difficult to create production-ready software.
- LLM limitations can be exacerbated by:
 - Lack of engineering rigor in prompt engineering.
 - Natural language can be ambiguous.
 - It is a newly created field.

Challenges in Building Production-Level Applications with LLM (2/2)

- Ambiguity occurs in the way prompts are written (by humans) and how they are interpreted (by LLMs). For example:
 - Ambiguous output format: downstream applications expect outputs in a certain format, which LLMs do not necessarily provide consistently.
 - Inconsistency in user experience: LLMs are stochastic; there is no guarantee that the model will provide the same output given the same input every time.

Areas of Enhancement

- Enhance context input into the model: give the model access to external data sources and tools for information gathering.
- Set up guardrails: protect systems and users.
- Add model router and gateway: support complex pipelines and security.
- Add cache: optimize for latency and cost.
- Add complex logic and write actions to maximize capabilities.

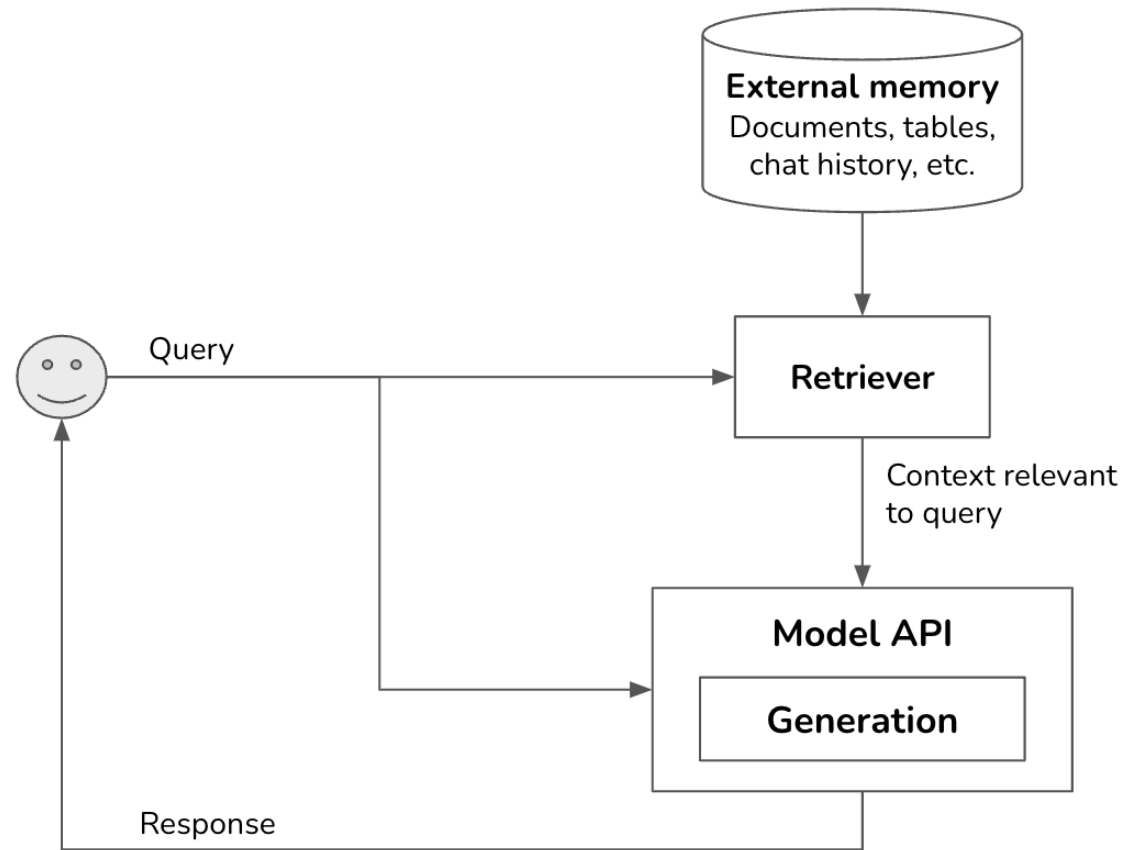
Performance-Driven Development

Although foundation models are a recent evolution in the field, the principles of building AI enterprise applications remain the same:

- Map business metrics to AI metrics.
- Systematic experimentation.
- Experiment with different prompts (equivalent to hyperparameter tuning).
- Optimize for performance, latency, and cost.
- Set up feedback loops to iteratively improve our applications.

Retrieval-Augmented Generation (RAG)

- Similar to feature engineering in ML, RAG augments each query with necessary information.
- Context construction: gather the relevant information for the query.
- The more context provided to the model, the less it needs to rely on its training.
- In-context learning delays a model from becoming outdated by continually incorporating new information.



Add Guardrails (1/3)

Input guardrails

- The risk of exposing sensitive or private data to third-party model APIs arises.
- Some guardrails include obfuscating personal information (ID numbers, phone, bank accounts, etc.), human faces, specific labels, keywords, and phrases that identify sensitive information.

Add Guardrails (2/3)

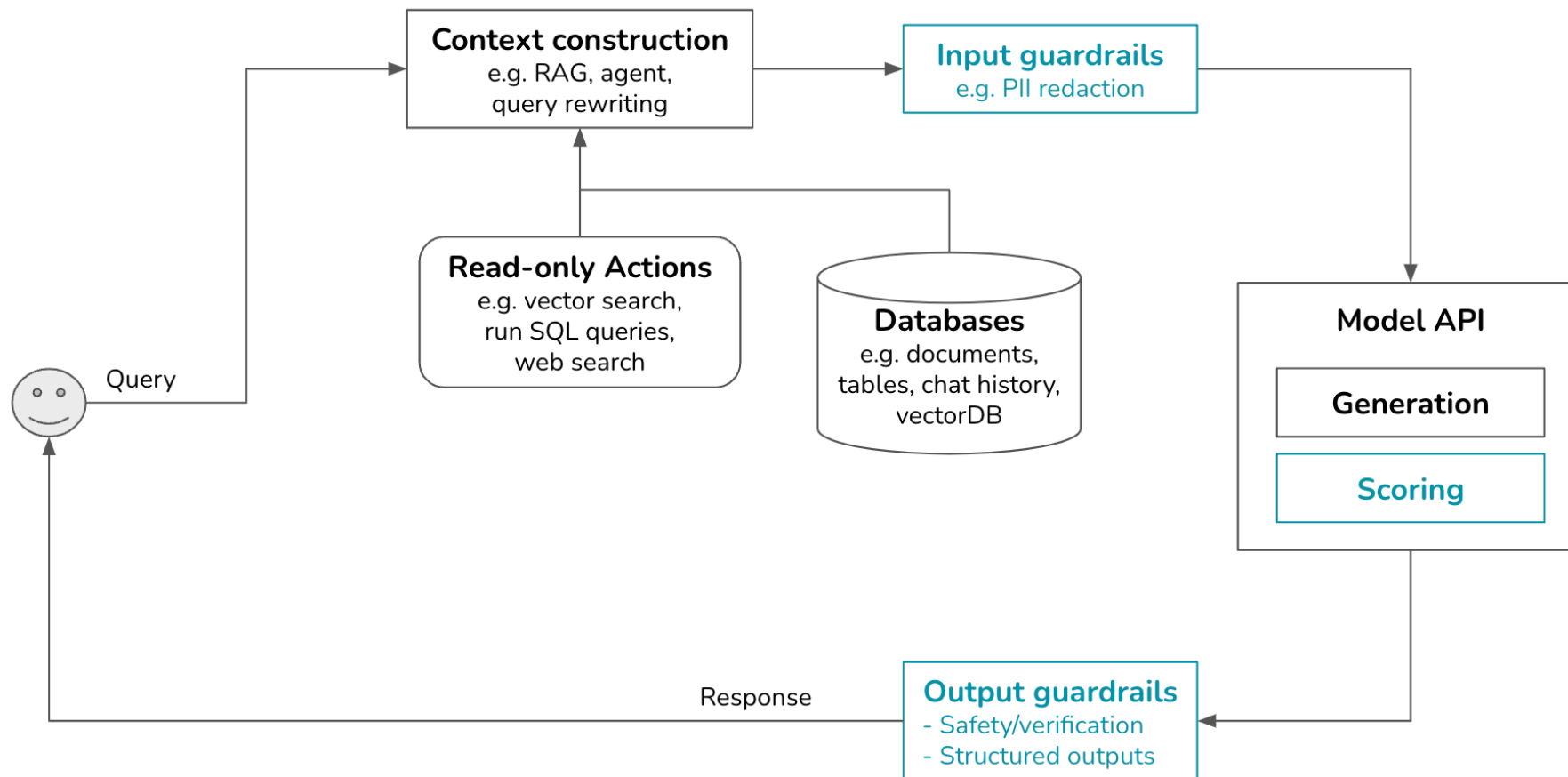
Model jailbreaking

- Preclude the model from executing queries that can be harmful.
- Ex.: no SQL queries.

Add Guardrails (3/3)

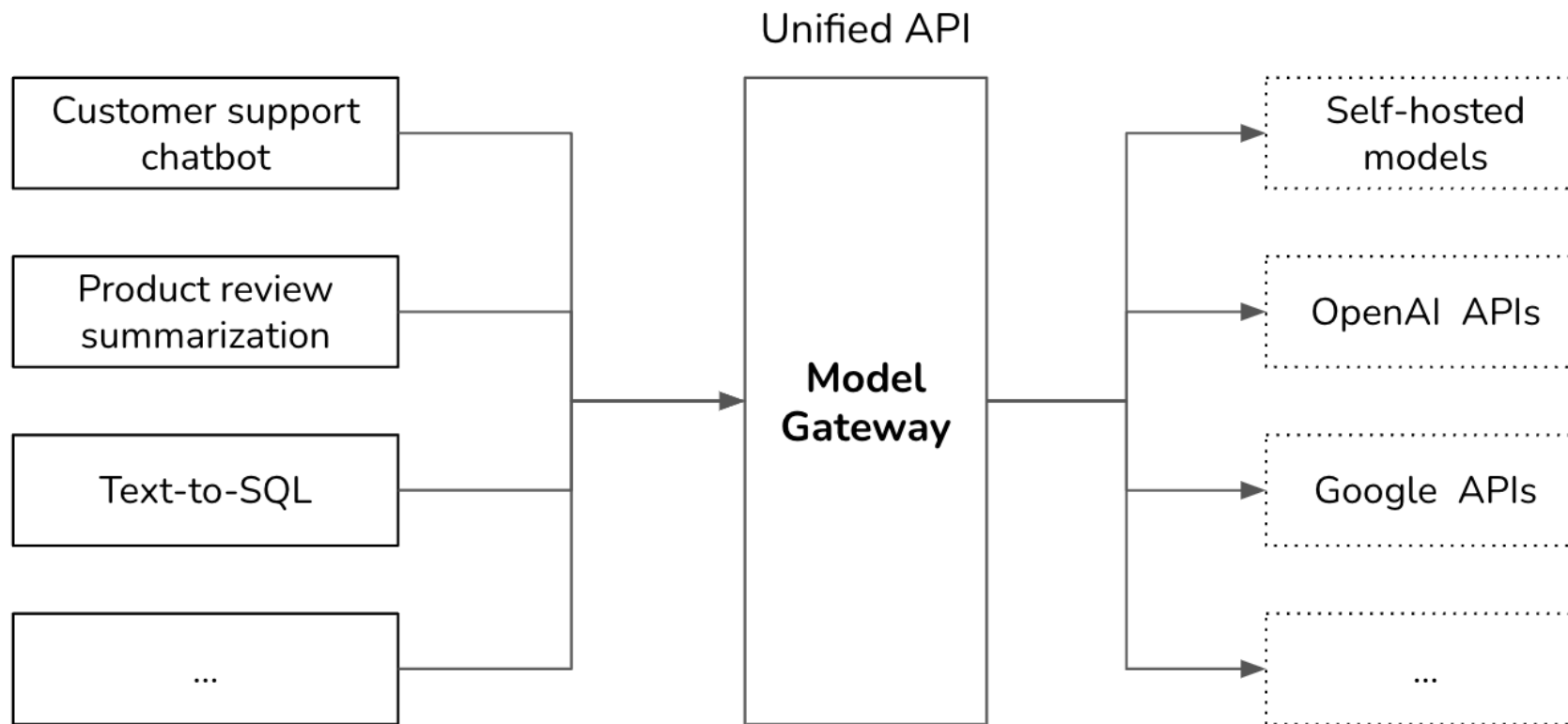
Output guardrails

- Evaluate the quality of each generation, including empty responses, malformed responses, toxic responses, factually inconsistent responses, responses that contain sensitive information, and brand-risk responses.
- Specify the policy to deal with different failure modes.



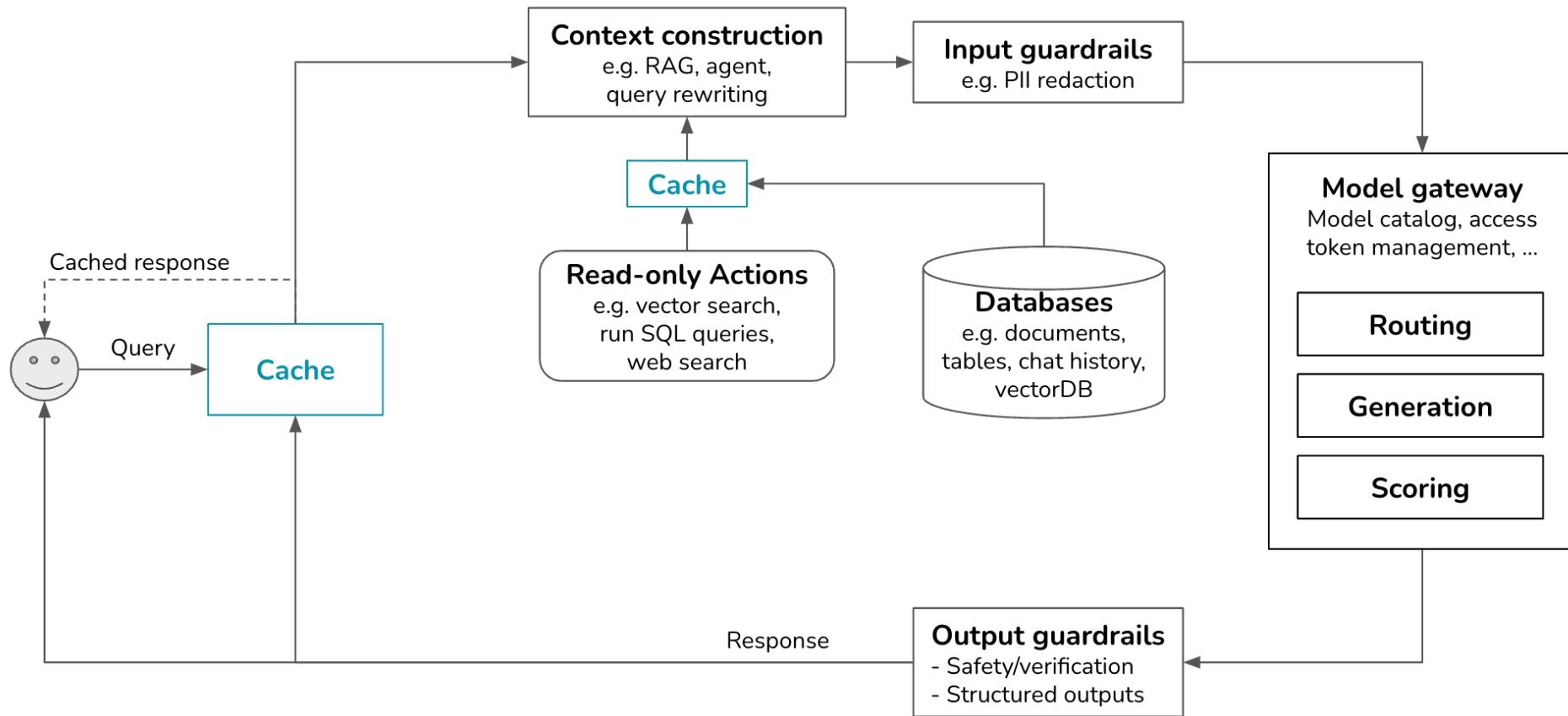
Add Model Router and Gateway

- A model router selects the best suitable model for the job:
 - An intent classifier predicts what the user is trying to do.
 - The right model is chosen for the task based on the predicted intent.
- An intent classifier can also prevent out-of-scope conversations.
- A model gateway allows the system to interface with different models in a unified and secure manner.



Caching

- Caching can significantly reduce latency and cost.
- Prompt cache:
 - Store overlapping segments for reuse.
 - Useful for applications with long system prompts or that involve long documents.
- Exact cache:
 - Cache stores processed items for reuse later.
 - Can be used to reduce vector search in embedding-based retrieval.
- Semantic cache:
 - Determines semantic similarity between queries.



Main Points

Main Points

1. Production AI applications require far more engineering than a working prototype: robust evaluation, guardrails, routing, caching, and feedback loops are all necessary.
2. The AI engineering stack evolves in layers: embedded software → custom API calls → RAG → guardrails → model routing → caching — each layer adds capability and control.
3. Guardrails operate on both inputs (blocking sensitive data or out-of-scope queries) and outputs (catching empty, malformed, toxic, or factually inconsistent responses).
4. Model routers use intent classification to direct queries to the most appropriate model, balancing quality, latency, and cost across a portfolio of models.

Main Points (cont.)

5. Caching — prompt, exact, and semantic — can substantially reduce both latency and cost, especially in applications with long system prompts or repetitive queries.
6. Business metrics must be mapped to AI metrics; systematic experimentation, prompt tuning, and feedback loops are the levers for iterative improvement.
7. In-context learning via RAG reduces model staleness by continuously incorporating new information without requiring retraining.

References

References

- Huyen, Chip. AI engineering: Building applications with foundation models. O'Reilly Media, Inc., 2024.
- Huyen, Chip. Designing machine learning systems. O'Reilly Media, Inc., 2022.